Technical Notes: Verilog RTL and RISC-V Practice Activities

Day 2: FSM controllers and memory design

2.1 Sequence detector (1011, overlapping)

- **Approach:** Two FSMs, a Moore version and a Mealy version, each built from a state register and combinational next-state logic, with the outputs decoded from the state (Moore) or from the state and input (Mealy). Overlap is handled by returning to the state that matches the longest valid suffix after a match.
- **Verification:** At least three input streams, including overlapping patterns such as 1011011.
- **Result:** Both detectors flagged the same matches.
- **Timing note:** The Mealy output depends on the current input and state, so it asserts in the same cycle the last 1 arrives. The Moore output depends only on the state, so it asserts one clock later.
- **Issues:** No significant issues recorded.

2.2 Vending machine controller

- **Approach:** Accumulator-based design. The accumulated total (0, 5 or 10) acts as the state. Reaching 15 dispenses immediately; 10 + 10 dispenses and returns 5 as change; cancel refunds the accumulated amount and clears it. Dispense and refund are one-cycle pulses.
- **Verification:** Directed testbench for the main payment, change and cancel scenarios. It does not exercise every transition in the table in vending_atm_states.md (for example, cancel from every possible total).
- **Result:** The scenarios that were tested behaved as specified.
- **Issues:** No significant issues recorded.

2.3 Synchronous FIFO

- **Approach:** 16 x 8 memory with read and write pointers and a 5-bit occupancy count. full is count == 16 and empty is count == 0. Simultaneous read and write are both accepted and leave the count unchanged. Writes when full and reads when empty are rejected.
- **Verification:** Fill to full, drain to empty, simultaneous read/write, write-when-full, read-when-empty, and ordering across pointer wraparound.
- **Result:** All checks passed.
- **Issues:** My first test wrongly expected occupancy to stay at 16 after a read. The correct value is 15 after the first read. I corrected the test, and it now checks data ordering through the wraparound write.

2.4 Design review

- **Approach:** Reviewed both sequence detectors, the vending controller, the FIFO and the other Day 2 designs for inferred latches, missing defaults and incomplete branches.
- **Verification:** Read through each combinational block, then re-simulated.
- **Result:** Default next-state and output values are present in all combinational blocks. Registered state uses non-blocking assignments. Full list in day2_design_review.md.
- **Issues:** The ATM's locked output had multiple procedural drivers, so I derived it from the FSM state instead.

2.5 Traffic light with pedestrian request

- **Approach:** FSM with parameters for red, green and yellow durations. A pending-request bit latches the button outside the WALK state, is served after the current green phase, and clears when WALK starts, so requests during WALK are ignored.
- **Verification:** Short durations overridden through parameters to keep simulation fast; a waveform with a pedestrian cycle.
- **Result:** Pedestrian cycle appears after green; a request during WALK was ignored.
- **Issues:** No significant issues recorded.

2.6 ATM controller

- **Approach:** States WAIT_CARD, WAIT_PIN, MENU and LOCKED. A counter locks the machine on the third wrong PIN. A withdrawal above the balance pulses insufficient_funds; otherwise the balance is updated.

- **Verification:** Correct PIN, three wrong PINs, insufficient balance and a successful withdrawal.
- **Result:** All four scenarios passed.
- **Issues:** locked was moved to a state decode to avoid multiple drivers. balance_valid now pulses for a balance inquiry.

2.7 Round-robin arbiter

- **Approach:** Four requesters with a one-hot grant. The grant is held until that requester drops its request, then scanning starts from the next requester.
- **Verification:** The testbench releases and reasserts the four requests, counts grants per requester and checks that the counts are equal.
- **Result:** 3 grants per requester.
- **Issues:** No significant issues recorded.

2.8 RAM, ROM and watchdog timer

- **Approach:** Single-port synchronous 16 x 8 RAM with read-before-write behaviour on a same-address edge. ROM loaded with $readmemh. Watchdog counter reloads on kick and emits a one-cycle reset pulse on timeout, with a programmable timeout.
- **Verification:** One combined testbench checks RAM write/read, ROM contents against rom.hex, a watchdog timeout, and a kick that prevents the timeout.
- **Result:** All checks passed.
- **Issues:** No significant issues recorded.